
BAB 9

ALGORITMA *CLUSTERING* K-MEANS DAN DBSCAN

9.1 TUJUAN PERCOBAAN

Setelah mengikuti praktikum ini, mahasiswa diharapkan mampu:

1. Memahami konsep algoritma *clustering* K-Means dan DBSCAN
2. Mengimplementasikan algoritma *clustering* K-Means dan DBSCAN dari sebuah *dataset*
3. Menampilkan visualisasi *cluster* yang terbentuk

9.2 ALAT-ALAT YANG DIGUNAKAN

Pada percobaan ini, akan digunakan berbagai alat dan bahan yang mendukung kegiatan praktikum. Alat-alat tersebut meliputi:

- Daftar alat utama: OS Windows/Linux/macOS, Anaconda, Google Colab
- Pustaka: *pandas*, *numpy*, *sklearn*

9.3 DASAR TEORI

9.3.1 *CLUSTERING*

Clustering merupakan proses pembentukan *cluster* atau klaster/kelompok dari sekumpulan data, dan data-data yang berada pada satu *cluster* tertentu memiliki kemiripan yang tinggi, sedangkan data-data yang berada pada *cluster* yang berbeda memiliki kemiripan yang rendah.

Pengelompokan data pada proses *clustering* didasarkan pada suatu kriteria pengelompokan, yaitu kemiripan data. Kemiripan data dapat diukur dengan beberapa cara, salah satunya adalah menggunakan ukuran jarak. Dengan menggunakan ukuran jarak, maka data yang berada dalam satu *cluster* memiliki jarak yang kecil, sedangkan data pada *cluster* yang berbeda memiliki jarak yang besar. Ukuran jarak yang dapat digunakan pada *clustering* adalah Euclidean distance. Persamaan Euclidean distance adalah:

$$d = \sqrt{\sum n_i (x_i - y_i)^2}$$

Apabila data berupa kategori, maka perhitungan jarak dapat menggunakan Simple Matching Distance. Simple Matching Distance biasa disebut juga dengan Hamming Distance. Persamaan jarak Simple Matching Distance adalah:

$$\delta(x, y) = \begin{cases} 0 & \text{if } x = y \\ 1 & \text{if } x \neq y \end{cases}$$
$$d_{sim}(x, y) = \sum_{j=1}^d \delta(x_j, y_j)$$

9.3.2 K-MEANS

K-Means merupakan algoritma *clustering* non-hierarki yang bertujuan membagi sekumpulan data menjadi *k cluster*. Tahapan-tahapan dalam algoritma K-Means adalah:

1. Tentukan banyaknya *cluster/kelompok*
2. Alokasikan data ke dalam kelompok secara acak.
3. Hitung pusat *cluster* dari data yang ada di masing-masing *cluster*.
4. Alokasikan masing-masing data ke *centroid* terdekat.

5. Kembali ke langkah 3, jika masih ada data yang berpindah *cluster* atau ada perubahan nilai *centroid* di atas nilai ambang/Threshold yang ditentukan, atau jika perubahan nilai pada fungsi objektif masih di atas nilai ambang atau dengan batasan iterasi max.
6. Selesai

9.3.3 DBSCAN

Algoritma *Density-Based Spatial Clustering of Applications with Noise* (DBSCAN) adalah algoritma *clustering* yang berbasis kepadatan. Algoritma ini bekerja dengan mengelompokkan titik-titik data yang rapat ke dalam *cluster* dan mengidentifikasi titik-titik yang tersebar (*outliers*) sebagai *noise*. DBSCAN tidak memerlukan jumlah *cluster* yang ditentukan sebelumnya dan dapat menemukan *cluster* dengan bentuk yang tidak teratur. Tahapan dari algoritma DBSCAN adalah:

1. Menentukan 2 parameter parameter utama yaitu jarak maksimum antara dua titik yang (epsilon - ϵ) dianggap sebagai tetangga dan jumlah minimum titik yang harus berada dalam radius epsilon agar suatu titik dianggap sebagai titik inti atau core point (minimum points - MinPts) .
2. Mengidentifikasi Core Points.
Suatu titik dianggap sebagai core point jika terdapat MinPts atau lebih titik dalam radius epsilon dari titik tersebut.
3. Perluasan *Cluster*
 - Dimulai dari sebuah core point yang belum terklaster.
 - Memperluas klaster dengan mencari semua titik yang terhubung dengan titik inti tersebut melalui titik-titik core dan border (titik yang berbatasan dengan core point tetapi bukan core point sendiri).
 - Proses ini diulangi hingga semua titik yang terhubung dengan titik inti awal telah terklaster.
4. Penanganan *Noise*.
Titik yang tidak termasuk dalam *cluster* mana pun akan dianggap sebagai *noise*.

9.4 PROSEDUR PERCOBAAN

9.4.1 IMPOR DATA

Praktikum kali ini menggunakan *dataset* bunga Iris dari UCI Machine Learning Repository. Unduh *dataset* yang akan digunakan pada praktikum kali ini. Anda dapat menggunakan aplikasi wget untuk mengunduh *dataset* dan menyimpannya dalam Google Colab. Jalankan *cell* di bawah ini untuk mengunduh *dataset*.

```
1 ! wget https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data
```

Setelah *dataset* berhasil diunduh, langkah berikutnya adalah membaca *dataset* dengan memanfaatkan fungsi `readcsv` dari *library* `pandas`. Lakukan pembacaan berkas CSV ke dalam *DataFrame* dengan nama data menggunakan fungsi `readcsv`. Jangan lupa untuk melakukan import *library* `pandas` terlebih dahulu.

```
1 import pandas as pd
2 data=pd.read_csv('iris.data', header=None, names=['sepal_length','sepal_width',
    'petal_length', 'petal_width', 'species'])
```

Cek isi *dataset* Anda dengan menggunakan perintah `head()`

```
1 data.head()
```

9.4.2 PREPROCESSING

Sebelum dilakukan proses *clustering*, perlu dilakukan beberapa tahapan pemrosesan awal data. Tahapan pertama adalah memisahkan kelas data dari *DataFrame*, karena algoritma K-Means tidak memerlukan informasi kelas. Pisahkan kelas data ke dalam sebuah variabel bernama `kelas`.

```
1 kelas=data.pop('species')
```

Pada praktikum kali ini, kita akan memvisualisasikan data hasil *clustering* menggunakan scatter plot. Scatter plot hanya dapat digambarkan dalam bentuk 2 dimensi, dengan demikian data yang digunakan juga hanya terdiri dari 2 fitur. Hapus fitur `petal_length` dan `petal_width` pada data, sehingga fitur yang digunakan adalah `sepal_length` dan `sepal_width`.

```
1 if 'petal_length' in data.columns:
2     del data['petal_length']
3 if 'petal_width' in data.columns:
4     del data['petal_width']
```

Tampilkan data setelah penghapusan fitur `petal_length` dan `petal_width`

```
1 data
```

Salah satu tahap pada algoritma *clustering* (misal K-Means) adalah perhitungan jarak Euclidean. Agar tidak ada fitur yang mendominasi pada perhitungan jarak, maka perlu dilakukan normalisasi terlebih dahulu agar rentang nilai setiap fitur sama. Implementasikan metode normalisasi Min-Max menggunakan fungsi bernama `minmax` pada *cell* di bawah ini.

```
1 def minmax(df_input):
2     list_fitur=df_input.columns[:-1]
3     for fitur in list_fitur:
4         max=df_input[fitur].max()
5         min=df_input[fitur].min()
6         df_input[fitur]=(df_input[fitur]-min)/(max-min)
7     return df_input
```

Lakukan normalisasi data dan simpan hasilnya pada variabel bernama `data_normal`

```
8 data_normal=minmax(data)
```

Tampilkan isi *DataFrame* `data_normal`

```
9 data_normal
```

9.4.3 VISUALISASI DATA

Visualisasi data dilakukan untuk memahami struktur dari data. Informasi ini dapat digunakan untuk

mengetahui cara pembentukan *cluster* yang baik. Visualisasikan data_normal menggunakan *scatter plot*. Gunakan *library* matplotlib.pyplot.

```
1 import matplotlib.pyplot as plt
2
3 warna={'Iris-setosa':'red','Iris-versicolor':'green', 'Iris-virginica':'blue'}
4 plt.scatter(data_normal['sepal_length'],data_normal['sepal_width'],c=kelas.map(
    warna))
5 plt.plot()
```

Fungsi *scatter* pada matplotlib menerima input berupa nilai variabel x, nilai variabel y, dan informasi kelas yang akan digunakan untuk membedakan warna elemen *scatter plot*.

9.4.4 K-MEANS CLUSTERING

Sebelum mengimplementasikan algoritma K-Means, perlu dilakukan implementasi perhitungan jarak. K-Means pada praktikum ini menggunakan perhitungan jarak Euclidean. Implementasikan perhitungan jarak Euclidean menggunakan fungsi bernama *euclidean* pada *cell* di bawah ini.

```
1 def euclidean(data1,data2):
2     jarak=np.square(data1-data2)
3     jarak=np.sum (jarak)
4     return np.sqrt(jarak)
```

Langkah selanjutnya adalah implementasi algoritma K-Means. Tahapan-tahapan dalam algoritma K-Means adalah:

1. Penentuan keanggotaan *cluster* secara acak
2. Hitung *centroid* berdasarkan data yang ada di masing-masing *cluster*
3. Alokasikan masing-masing data ke *centroid* terdekat
4. Kembali ke langkah 2 selama kondisi berhenti belum terpenuhi. Iterasi berhenti jika salah satu kondisi berikut terpenuhi:
 - a. Tidak ada perubahan keanggotaan *cluster*
 - b. Iterasi melebihi parameter *iter_max*

Implementasikan algoritma K-Means menggunakan fungsi bernama *kmeans* pada *cell* di bawah ini. Fungsi *kmeans* memiliki 3 parameter input, yaitu *data*, *K*, dan *iter_max*:

```
1 from random import randint
2 import copy
3 import numpy as np
4
5 def kmeans(data_input,k,iter_max=200):
6     n_data=data_input.shape[0]
7     n_fitur=data_input.shape[1]
8     iter=1
9     cluster=np.array([randint(0,k-1) for i in range(n_data)])
10    centroid=np.zeros((k,n_fitur))
11    cluster_tidak_berubah=False
12    while not cluster_tidak_berubah and iter<iter_max:
13        jarak=np.zeros((n_data,k))
14        for i in range(k):
15            centroid[i,:]=data_input.iloc[np.where(cluster==i)[0]].mean().tolist()
```

```

16     for i in range(n_data):
17         for j in range(k):
18             jarak[i][j]=euclidean(data_input.iloc[i],centroid[j])
19     cluster_baru=[np.argmin(jarak[i]) for i in range(n_data)]
20     cluster_tidak_berubah=np.array_equal(cluster_baru,cluster)
21     cluster=np.array(copy.deepcopy(cluster_baru))
22     iter+=1
23     return cluster,centroid,jarak

```

Pada fungsi tersebut, variabel *cluster* berisi keanggotaan *cluster* dari masing-masing data. Variabel *cluster* awalnya bernilai acak dengan rentang 0 sampai dengan k-1. Variabel *centroid* merupakan centroid masing-masing *cluster* dengan struktur data berupa matriks berdimensi k × banyak fitur. Variabel *cluster_tidak_berubah* merupakan variabel Boolean yang menandakan apakah keanggotaan *cluster* yang terbentuk pada iterasi sekarang sama dengan keanggotaan *cluster* yang terbentuk pada iterasi sebelumnya.

Proses *clustering* terjadi di dalam *looping while*. *Looping* akan terus dilakukan selama masih terjadi perubahan keanggotaan *cluster* (*cluster_tidak_berubah* = False) dan iterasi kurang dari iterasi maksimum (*iter*<*iter_max*).

Tahapan pertama dalam *looping* adalah menghitung *centroid*, yang merupakan rata-rata data pada masing-masing *cluster*. Selanjutnya, jarak setiap data dengan setiap *centroid* dihitung dan disimpan pada variabel jarak. Keanggotaan *cluster* yang baru diperoleh dari indeks jarak dengan nilai minimum dari masing-masing data ke *centroid* dan disimpan pada variabel *cluster_baru*. Setelah itu, dilakukan perbandingan dengan variabel *cluster*, untuk mengecek apakah ada perubahan keanggotaan *cluster* pada iterasi sebelumnya. Jika ada perubahan, variabel *cluster_tidak_berubah* berisi nilai False, sedangkan jika tidak ada perubahan maka variabel *cluster_tidak_berubah* berisi nilai True.

Lakukan proses *clustering* dengan memanggil fungsi *kmeans* dengan parameter input berupa *data_normal* dan *k=3*.

```

1 cluster=kmeans(data_normal,3)

```

Tampilkan keanggotaan *cluster* pada masing-masing data

```

1 print(cluster)

```

Tampilkan hasil *clustering* menggunakan scatter plot

```

1 import matplotlib.pyplot as plt
2 plt.scatter(data_normal['sepal_length'], data_normal['sepal_width'],
3             c=cluster[0])
3 plt.plot()

```

9.4.5 DBSCAN CLUSTERING

Pada praktikum ini, implementasikan algoritma DBSCAN menggunakan pustaka *sklearn*. Pada contoh ini terdapat dua parameter penting yang diatur pada program yaitu *eps* dan *min_sample*. *Eps* adalah jarak maksimum antara dua sampel agar satu sampel dianggap berada di sekitar sampel lainnya. Parameter ini harus dipilih dengan tepat untuk set data dan fungsi jarak. Nilai default dari parameter *eps* adalah 0.5. *Min_sample* adalah jumlah sampel (atau bobot total) di lingkungan sekitar agar suatu titik dianggap sebagai titik inti (termasuk titik itu sendiri). Jika *min_sample* ditetapkan tinggi, maka DBSCAN akan membentuk klaster yang lebih padat, sedangkan jika ditetapkan rendah, maka klaster yang terbentuk akan lebih jarang. Nilai *default* dari parameter *min_sample* adalah 5.


```
1 from sklearn.cluster import DBSCAN
2
3 epsilon = 0.11
4 min_samples = 5
5 dbscan = DBSCAN(eps=epsilon, min_samples=min_samples)
6 cluster = dbscan.fit_predict(data_normal)
```

Tampilkan keanggotaan *cluster* pada masing-masing data

```
1 print(cluster)
```

Tampilkan hasil *clustering* menggunakan scatter plot

```
1 import matplotlib.pyplot as plt
2 plt.scatter(data_normal['sepal_length'], data_normal['sepal_width'],
3             c=cluster)
3 plt.plot()
```

9.5 TUGAS

Pada tugas kali ini Anda diminta mengidentifikasi pasangan fitur apa yang menghasilkan *cluster* yang ideal pada K-Means dan DBSCAN menggunakan data Iris. Langkah yang harus dilakukan adalah:

1. Buatlah plot untuk setiap pasangan fitur yang mungkin.
2. Lihatlah plot yang terbentuk. Identifikasikan pasangan fitur apa yang dapat membedakan *cluster* dengan baik. Hal ini ditandai dengan plot yang memiliki warna-warna yang saling berpisah atau tidak bercampur.
3. Lakukan *clustering* menggunakan K-Means dengan $k=3$ dan DBSCAN. Lalu amati apakah *cluster* yang dihasilkan dapat terpisah dengan baik.

Petunjuk pengerjaan soal:

1. Buka tautan berikut, pastikan Anda login menggunakan **akun student UB**.
<https://s.ub.ac.id/nbkal09>
2. Klik tombol Copy to Drive
3. Beri nama file Praktikum 9 - Nama - NIM
4. Isilah *cell* yang kosong
5. Download file *.ipynb dengan cara
klik **File** → **Download .ipynb**
6. Kumpulkan file *.ipynb ke asisten

9.6 KESIMPULAN

Setelah melakukan praktikum ini mahasiswa diwajibkan untuk membuat kesimpulan.

Kesimpulan dari percobaan ini merupakan bagian penting untuk merangkum temuan utama dan relevansinya dengan tujuan percobaan. Mahasiswa harus mampu menyimpulkan:

- Apakah tujuan percobaan telah tercapai.

- Hubungan antara hasil percobaan dan dasar teori.
- Faktor-faktor yang memengaruhi hasil percobaan, termasuk kemungkinan kesalahan yang terjadi.

Kesimpulan harus disusun secara singkat, jelas, dan berdasarkan data yang diperoleh.